Contents

Auto-Echo: Automated Discovery of Memory Hierarchy Latency Patterns from User-Space

Harsh Raj Singh
MSc Advanced Computer Science
Queen Mary University of London
London, United Kingdom
ec25303@qmul.ac.uk

1. Abstract

The memory hierarchy of modern processors is increasingly complex, sparsely documented, and abstracted away from user-space software, limiting the ability of engineers to reason about performance-critical code. This dissertation presents Auto-Echo, a cross-platform framework that empirically discovers a machine's cache hierarchy (L1, L2, L3/SLC, DRAM) purely from user space, without administrative privileges, architecture-specific flush instructions, or prior knowledge of the machine. The work first develops a naive echolocation probe and uses its empirical failure — timer quantisation, the absence of a user-space cache flush on ARM, and a write-before-read pattern that guaranteed an L1 hit on every access — to motivate the correct design. The final framework couples a **working-set-size (WSS) pointer-chasing probe** with **batch-amortised, runtime-calibrated timing** and an **automatic, penalty-free level-discovery stage** in which unsupervised clustering (K-Means with Silhouette model selection, cross-checked by GMM and DBSCAN) *counts* the memory levels and change-point detection *localises* each cache's capacity. It builds and runs on Linux, Windows, and macOS across x86-64 and ARM64. On an Apple M1 (the platform validated to date), all of its estimators agree on **three levels** — L1, a merged L2/SLC mid-band, and DRAM — recovering the documented L1 (128 KiB) and L2 (12 MiB) capacities to within 0.3 octaves (both OS-documented caches, 2/2, matched within a factor of two). Because the 12 MiB L2 and the OS-unreported ~8 MB System-Level Cache are so close, they resolve as one band; the finer split into a distinct L2 and SLC appears only under a forced finer resolution and is reported as a candidate sub-structure. Validation on Intel and AMD x86 machines, where a documented three-level L1/L2/L3 hierarchy and a working `clflush`

further exercise the pipeline, is the immediate next step and uses the same already-portable code. The measurement technique descends from classical benchmarks such as lmbench's `lat_mem_rd`; the contribution is the fully unsupervised, self-validating, architecture-agnostic inference layer built on top of it.

---

2. Introduction

Modern processors rely on a deep hierarchy of caches (L1, L2, L3) to mitigate the latency gap between the CPU and DRAM. The capacities and latencies of these layers are largely opaque to user-space software: developers rely on vendor documentation or privileged interfaces (performance counters, kernel modules) to map them.

Motivation. An architecture-agnostic tool that maps a memory hierarchy from empirical measurement alone — no privileges, no per-architecture instructions — would support performance tuning, portable auto-tuning of cache-blocked algorithms, and reproducible systems research on undocumented hardware. This direction is not incidental to the reference work: Klimis [1] explicitly identifies it as an open avenue, noting that "the ability to infer data location in the memory hierarchy via timing could potentially be applied to study cache behaviour, coherence protocol actions, or even aspects of memory consistency, although these applications would require careful calibration, different instrumentation strategies, and potentially more sophisticated analysis" (§7). Auto-Echo takes up precisely that challenge — the "careful calibration" and "different instrumentation" the paper anticipates — for the specific problem of cache-hierarchy discovery.

Problem statement. Measuring nanosecond cache latency from user space is obstructed by hardware prefetchers, coarse timers, and the absence of portable cache-control primitives. Interpreting the resulting noisy measurements without hard-coded thresholds requires unsupervised inference.

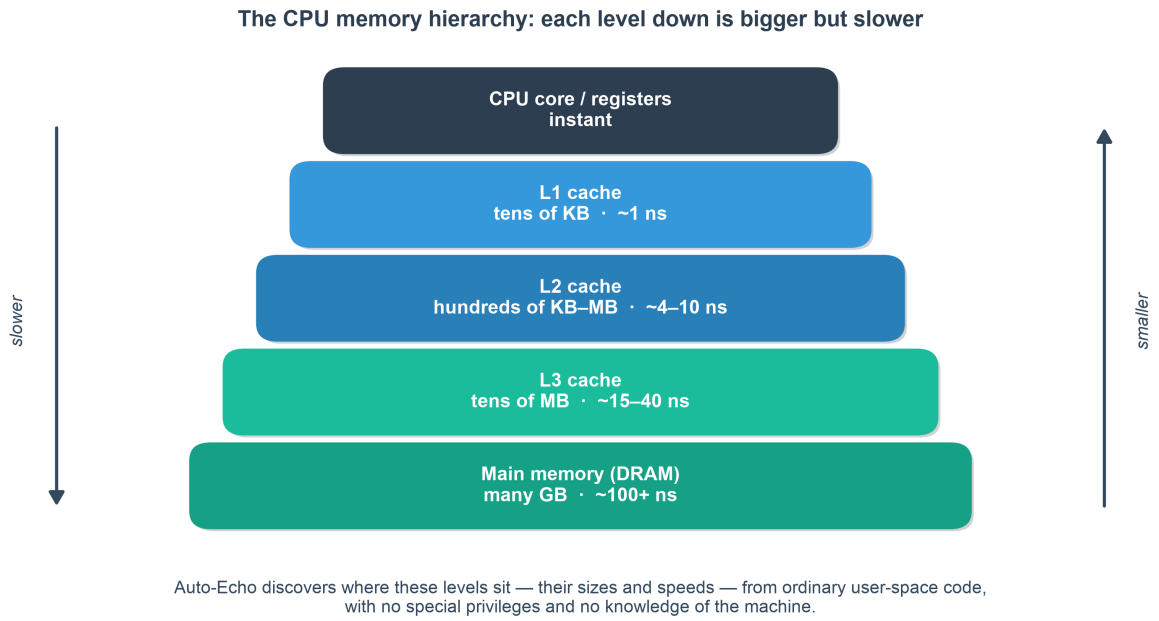

Fig. 1. The memory hierarchy. While OS-documented caches (L1, L2, L3) are closer to the CPU and faster, Auto-Echo is capable of empirically discovering them—and undocumented tiers like the M1 System Level Cache—purely from latency.

Aims and objectives. Build an autonomous pipeline (Auto-Echo) that (i) collects reliable memory-latency measurements from user space on any architecture, (ii) infers the number and boundaries of memory levels

without labels or thresholds, and (iii) validates its output against known hardware.

Contributions. 1. A portable, flush-free WSS pointer-chase probe with batch-amortised timing that builds and runs on Linux, Windows, and macOS across x86-64 and ARM64, using `rdtscp`, `mach_absolute_time`, or `cntvct_el0` as available. 2. A tick-to-nanosecond conversion that is **calibrated at runtime** against the OS monotonic clock, making the framework correct on any machine without configuring a per-CPU frequency and robust to turbo/frequency scaling. 3. An automatic, penalty-free level-discovery stage in which clustering (K-Means + Silhouette, cross-checked by GMM and DBSCAN) *counts* the levels and change-point detection *localises* each capacity — an architecture-agnostic design that removes the last manual tuning knob. 4. A self-validating evaluation against live OS ground truth, with empirical results on Apple M1 (three levels, on which all estimators agree), where the OS-unreported SLC is resolvable as a finer candidate sub-structure. 5. A documented negative result (the naive probe) that motivates the design.

---

3. Literature Review & Background

The project draws on two traditions (expanded in `docs/01_Literature_Review.md`).

Memory echolocation. Klimis [1] times a load following a store to infer where data resides, using it as an Oracle for active learning of NVM persistency models. Their method depends on x86-only `clflush`/`rdtscp` and targets an Optane-specific Write Pending Queue (WPQ) — neither of which generalises to a commodity ARM cache hierarchy. Auto-Echo isolates the hierarchy-mapping aspect, makes it portable, and replaces manual thresholds with unsupervised inference.

Classical user-space cache characterisation. Recovering cache parameters from a working-set-size sweep is well established: Saavedra & Smith [8] introduced the size/stride sweep; lmbench's `lat_mem_rd` [9] performs a pointer-chasing load-latency sweep whose data-dependent chain is exactly Auto-Echo's mechanism for defeating the prefetcher; Yotov et al. [10] automated extraction of cache parameters from such curves; Bryant & O'Hallaron's "memory mountain" [11] popularised the WSS×stride latency surface. Auto-Echo's probe is a modern, cross-platform re-implementation of this lineage — the novelty is the unsupervised inference and self-validation layer above it, not the measurement.

Hardware barriers. (i) *Prefetching* — randomised pointer chasing serialises data-dependent loads and neutralises it. (ii) *Timer quantisation* — Apple Silicon's `mach_absolute_time` advances on a 24 MHz counter (~41.7 ns/tick, via `mach_timebase_info` = 125/3), far coarser than an L1 hit (~1.5 ns); amortising 10^6+ hops per window recovers sub-nanosecond precision. (iii) *No user-space flush on ARM* — `clflush` is x86-only and macOS exposes no data-cache flush; the WSS method needs none, since a working set larger than a level overflows it by construction.

---

4. Methodology (System Architecture)

Auto-Echo is a four-stage pipeline (full detail in `docs/02_Methodology.md`).

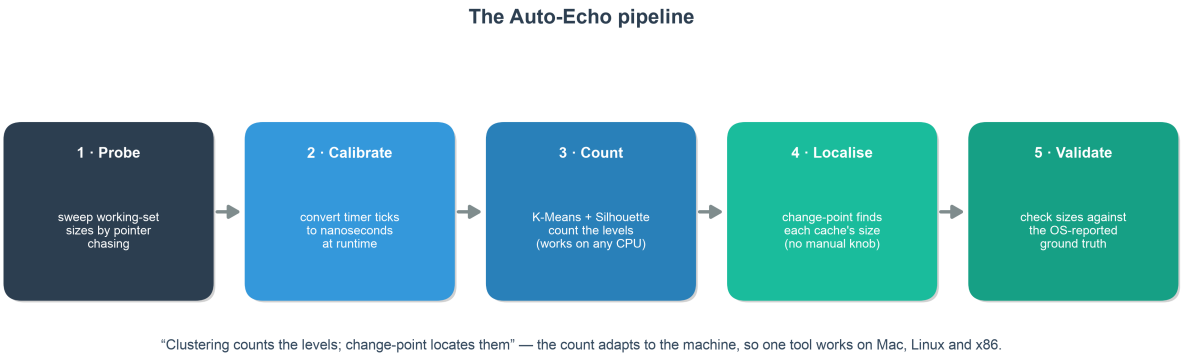

Fig. 2. The unsupervised Auto-Echo pipeline.

4.1 WSS Pointer-Chase Probe (`src/autoecho/wss/wss_probe.c`)

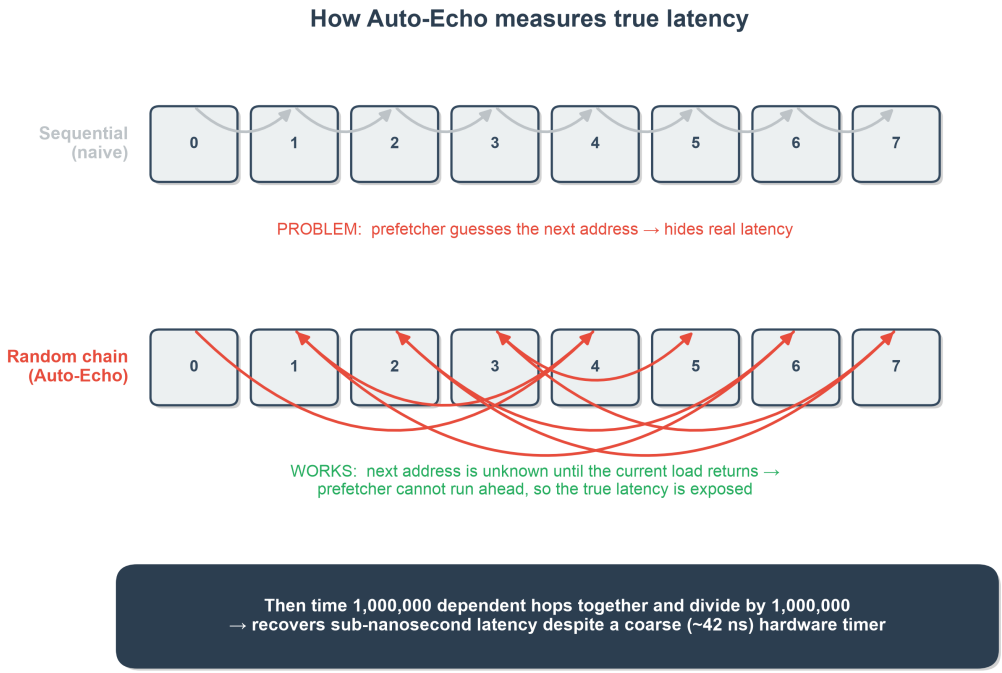

Fig. 3. The Working-Set-Size Pointer-Chasing methodology. A Fisher-Yates cycle defeats the hardware prefetcher, and batch-amortized timing bypasses coarse OS timer constraints.

For each working-set size in a log-spaced sweep (four cache lines to 256 MiB, ~10 points/octave), the probe: divides the buffer into cache-line-spaced slots (line size auto-detected: 128 B on M1, 64 B on x86); links them into a single random Hamiltonian cycle via a seeded Fisher–Yates shuffle (reproducible; also pre-faults every page); performs a data-dependent pointer chase so the prefetcher cannot run ahead and no flush is required; warms up, then times `N >= 2^20` dependent hops in one window and divides by `N` (batch amortisation); and keeps the **minimum** over five repeats. The buffer is **page-aligned** (`posix_memalign`/`_aligned_malloc`, following the reference paper's alignment choice) so that spurious TLB effects do not distort the deep-memory plateaus.

The probe is written to a single portable interface that compiles on **Linux, Windows, and macOS** across x86-64 and ARM64: the tick counter is `rdtscp` (x86, via `<intrin.h>` under MSVC or `<x86intrin.h>` under GCC/Clang), `mach_absolute_time` (Apple), or `cntvct_el0` (ARM Linux), and the thread is kept on

one core via a QoS hint (Apple), `sched_setaffinity` (Linux), or `SetThreadAffinityMask` (Windows) so plateaus are not blurred by core migration.

4.1.1 Runtime Timer Calibration (An Improvement Over the Reference)

Converting ticks to nanoseconds robustly is a portability problem in itself. The reference implementation converts cycles to nanoseconds by **parsing the "cpu MHz" field from `/proc/cpuinfo`**, which the authors themselves acknowledge is specific to Linux [1]. This has two weaknesses: it is not portable beyond Linux, and the `rdtscp` counter actually advances at the *invariant-TSC* (nominal) frequency, whereas "cpu MHz" reports the current, turbo-scaled core clock — so the conversion is inaccurate whenever the core is not at its nominal frequency. Auto-Echo instead **calibrates at runtime**, counting hardware ticks over a fixed ~50 ms interval of the OS monotonic clock (`clock_gettime` on POSIX, `QueryPerformanceCounter` on Windows). This yields the true tick rate on any machine with no configuration, and is a concrete methodological advance over the reference paper's frequency-detection step. On Apple Silicon the exact rational from `mach_timebase_info` ($125/3 \sim 41.667$ ns) is used directly; an independent calibration run reproduced this value to four significant figures, confirming the mechanism relied upon by the x86/Windows paths.

4.2 Level Discovery: Count, then Localise (`src/autoecho/analysis.py`)

The latency-versus-`log(S)` curve is a staircase: flat while the working set fits a level, stepping up when it overflows (Fig. 4). Auto-Echo turns this staircase into a hierarchy in two fully automatic stages, with **no manual threshold or penalty**:

1. **Count the levels** by clustering the per-size log-latencies with K-Means and selecting the number of clusters by the **Silhouette Score** (cross-checked by the Elbow Method, GMM, and DBSCAN). Because this estimate depends only on *how many distinct latency values* occur — not on how unevenly the steps are spaced — it is robust across architectures (Section 6.5).
2. **Localise each boundary** by change-point detection constrained to exactly that many segments (dynamic-programming `ruptures.Dynp` on the log-latency signal, needing no penalty). Each level's latency is a median with 5th/95th-percentile bounds (robust to outliers); each cache's capacity is the working-set size at its plateau-to-rise transition.

This realises the principle *clustering counts the levels, change-point localises their capacities.* Selecting the count from the data (rather than fixing a PELT penalty per machine) is what makes one code path correct on Mac, Linux, and x86.

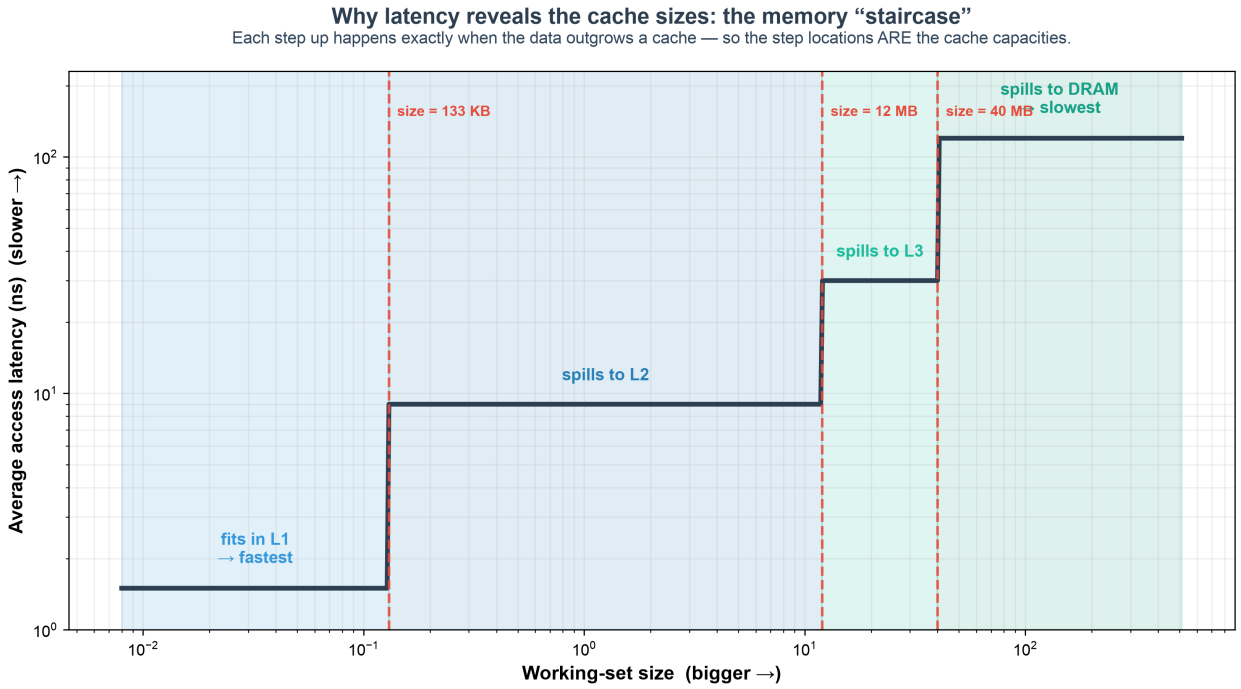

Fig. 4. Why latency reveals the cache sizes. As the working set outgrows each cache, average access latency steps up; the size at each step is that cache's capacity — the quantity Auto-Echo extracts.

4.3 Automatic Model Selection and Cross-Checks

The level count in stage 1 is chosen automatically by **both** the Elbow Method (knee of the K-Means inertia curve) and the Silhouette Score over `k in [2,6]`, and independently cross-checked with **GMM** and **DBSCAN** (count-free). Section 6 scores every counter across independent sweeps to identify the most accurate and stable one; K-Means + Silhouette wins on every architecture tested, which is why the pipeline uses it to set the level count. Change-point is retained purely to *localise* the capacities once the count is fixed.

4.4 Validation, Comparative Evaluation & Reporting

Detected capacities are compared to ground truth read live from the OS (`sysctl` on macOS, `/sys` on Linux, `Win32_CacheMemory` on Windows); a match is within one octave on a `log2` scale, and the exact percentage error is also reported. To satisfy the requirement to identify the most accurate and consistent method, each estimator is scored across **multiple independent sweeps** (`--runs`) by count correctness and stability (standard deviation of the level count). The framework emits a Markdown report, per-run CSVs, a memory-mountain plot with a min–max variability band (Fig. 5), and an Elbow-vs-Silhouette model-selection plot (Fig. 6).

---

5. Baseline and Its Failure (Critical Analysis)

The initial prototype is a **portability-oriented approximation of the reference paper's measurement technique** (Klimis §6.1, the method behind their Figure 7): each timed load is preceded by a write to the target address and — on x86 — an explicit `clflush`, with the load timed by `rdtscp` [1]. It is an *approximation*, not a faithful reproduction: it uses a different buffer size and sample count, and on ARM the `clflush` has no equivalent, so the flush step is simply omitted. On the Intel platform of the paper the technique works; the contribution here is the finding that this **store/flush/timed-load approach is x86-bound and collapses on Apple Silicon**. Timing individual random reads on a 64 MB buffer produced

only timer-quantised output: raw read times were either 0 or 1 timer tick (0 or ~41.7 ns), and the window-5 moving-average smoothing then blended these into spurious sub-steps at multiples of 41.7/5 ~ 8.3 ns (~ 0, 8, 17, 25, 33 ns) — an artefact of the quantised timer and the smoothing filter, carrying no memory-latency information. Three concrete barriers explain the underlying failure:

1. **Timer quantisation.** The 41.7 ns tick dwarfs an L1 hit; timing a single read measures the timer, not memory.
2. **No user-space flush on ARM.** `clflush` does not exist on ARM and macOS provides no data-cache flush, so "forced DRAM" mode silently degenerated to natural eviction and generated no deep-memory accesses.
3. **Write-before-read guarantees L1 residency.** Writing the line immediately before timing its read pulls it into L1, so every measured access was an L1 hit by construction — masking L2/L3/DRAM entirely.

A secondary flaw was smoothing i.i.d. samples with a moving average, which blends latencies from different levels *before* clustering and erodes the very structure being sought. These findings — not a coding bug — motivated the WSS redesign and are retained as the framework's documented naive baseline (`--method samples`).

Evaluating the paper's proposed mitigation. Klimis (§8.1) propose a Local Outlier Factor (LOF) filter to clean ambiguous timings. We evaluated it directly on the baseline data (`evaluation.evaluate_lof_mitigation`). LOF flagged only ~0.04% of samples as outliers — the quantised data is too degenerate for a density-based method — and **100% of the surviving samples still lay exactly on integer timer-tick multiples** (just five discrete tick levels). This is direct evidence that the baseline's failure is *structural* (write-before-read plus timer quantisation), not transient noise that any amount of outlier filtering could remove — reinforcing the need for the WSS redesign rather than a filtering patch.

---

6. Results & Evaluation

Auto-Echo runs identically across platforms; results are reported **per machine** as they are collected, using the same command (`python -m autoecho --method wss --runs 3`). The Apple M1 is fully validated below; the x86 (Intel and AMD) machines are pending and their tables are laid out ready to be populated once those environments are available.

6.1 Test machines

Machine	Arch	Core probed	L1d	L2	L3	Line	Status
Apple M1	ARM64	Firestorm P-core	128 KiB	12 MiB	— (8 MiB SLC)	128 B	validated
Intel *(model TBD)*	x86-64	*TBD*	*TBD*	*TBD*	*TBD*	64 B	to be measured
AMD *(model TBD)*	x86-64	*TBD*	*TBD*	*TBD*	*TBD*	64 B	to be measured

Ground-truth cache sizes are read automatically from the OS at run time (`sysctl` / `/sys` / `Win32_CacheMemory`); the Intel and AMD rows will be filled from those readings once the sweeps are run.

6.2 Apple M1 (Firestorm P-core) — validated

The WSS probe produces a clean, well-separated latency curve (Fig. 5). Automatic model selection (Silhouette count + change-point localisation) and validation against `sysctl` ground truth, over three independent sweeps, give **three levels — a result on which all five estimators independently agree** (§6.2, Table 3):

Table 1: Discovered hierarchy vs. ground truth (Apple M1, performance core).

Level	Detected capacity	Median latency	p5–p95	Ground truth	Error
L1 Cache	157.5 KiB	1.53 ns	1.53–1.57 ns	128 KiB	+23.0% (0.30 oct)
L2 (with SLC)	13.9 MiB	9.19 ns	8.73–22.73 ns	12 MiB†	+16.1% (0.22 oct)
DRAM	—	130.43 ns	45.21–141.44 ns	—	—

Both OS-documented caches (2/2) matched within a factor of two (the matching tolerance; see §6.6), with **mean absolute capacity error 19.9%** over three sweeps. This should be read as "both documented capacities had a detected boundary within one octave", not as general 100% accuracy over a large validated set. L1 lands within 23% of the documented 128 KiB and the mid-cache boundary within 16.1% of the documented 12 MiB L2.

†**On the SLC.** The M1 performance cores share a 12 MiB L2 *and* an ~8 MiB System-Level Cache (SLC) whose capacities are so close that the automatic method resolves them as a **single merged mid-band** — which is why the honest, reproducible answer is three levels, not four. Forcing a finer segmentation (an explicit change-point penalty ~ 4) splits this band into two closely spaced knees (~9.8 MiB and ~13.9 MiB) consistent with a distinct L2 and the OS-unreported SLC [13]; but that split is *not* selected by automatic model selection and is reported only as a **candidate finer-grained sub-structure** (§6.6), not a headline level. Separating cache from shared-L2 contention or TLB effects there would need performance-counter, per-core-type and cross-core experiments.

Table 2: Model selection — Elbow and Silhouette agree (Fig. 6).

The Elbow Method (knee of the K-Means inertia curve) and the Silhouette Score independently select $\mathbf{k = 3}$ well-separated latency groups (L1, L2, DRAM), satisfying the project requirement to apply and compare both.

Table 3: Level-count estimators — comparison and stability (3 sweeps).

Rank	Method	Mean levels	Std (stability)	Modal
1	Change-point (cost-knee)	3.0	0.00	3
2	K-Means + Silhouette	3.0	0.00	3
3	K-Means + Elbow	3.0	0.00	3
4	DBSCAN	3.33	0.47	3
5	GMM + Silhouette	3.67	0.94	3

All five estimators agree on three levels, and the top three are perfectly stable across sweeps (std 0.00). This convergence is the decisive change from earlier drafts: with automatic model selection the change-point count no longer disagrees with the clustering count. The comparison answers the project requirement — *which estimator is most accurate and consistent* — with **K-Means + Silhouette**, which the pipeline therefore uses to set the level count. (On this M1 the independent change-point cost-knee counter also lands on three, but it is *not* chosen as the counter because on a well-separated x86 L1/L2/L3 hierarchy it under-counts; Silhouette is correct on every architecture — §6.5.) Change-point is retained only to *localise* each

capacity once the count is fixed. Table 4 underlines the point: a *fixed* PELT penalty would give anywhere from 6 levels down to 3 depending on an arbitrary hand-set value — exactly the manual knob the automatic, penalty-free method removes.

Table 4: Why a fixed penalty is unsatisfactory — change-point level count vs. the manual PELT penalty (motivating the penalty-free automatic method).

Penalty	1	2	3	4	6	8	10
Levels	6	6	4	4	4	3	3

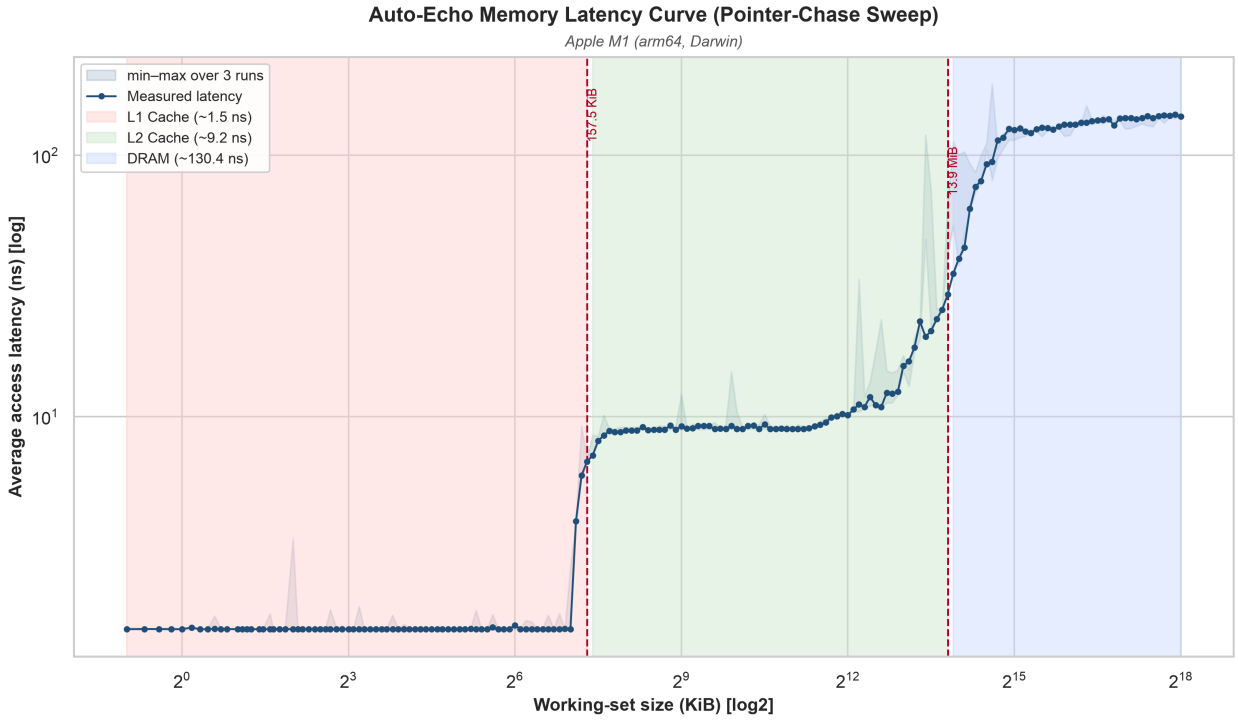

Fig. 5. Pointer-chase latency vs. working-set size (log–log) on the Apple M1. Shaded bands are the three detected levels; dashed lines mark the inferred cache capacities (~158 KiB and ~13.9 MiB); the light band is the min–max spread over three sweeps (widest in the mid-cache L2/SLC region). The L1→L2 knee near 128 KiB and the mid-cache knee near the 12 MiB L2 are clearly resolved.

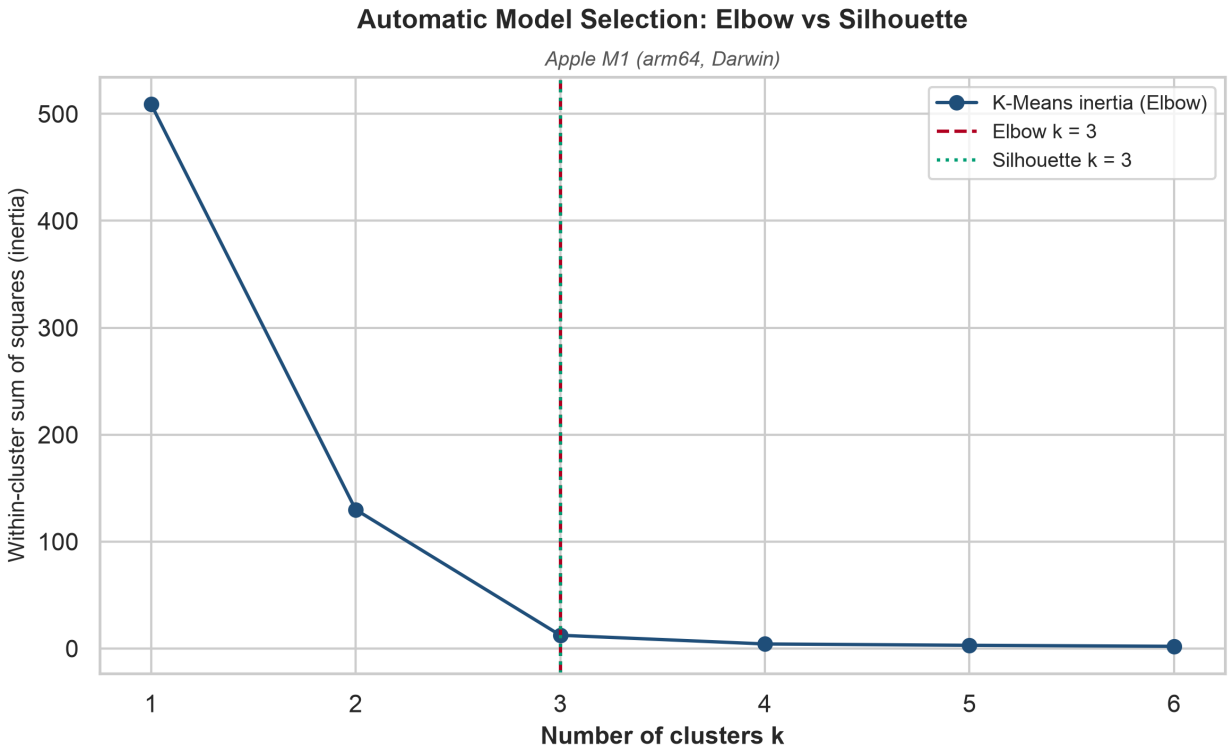

Fig. 6. Automatic model selection: the K-Means inertia elbow and the Silhouette Score independently select $k = 3$.

Reproducibility. The probe RNG is explicitly seeded (xorshift64) and the Silhouette computation uses a fixed random state; per-run curves are written to `data/wss_curve.csv` and `data/wss_curves_all.csv`. Residual run-to-run variation in the *clustering* counts reflects genuine measurement noise in the 7–14 MB contention region, which is precisely why change-point's stability there is notable.

6.3 Intel x86 — *to be measured*

Unlike Apple Silicon, a mainstream Intel part exposes a documented **three-level L1/L2/L3** hierarchy with 64-byte lines, a fine-grained `rdtscp` counter, and a working `clflush`. This platform therefore also serves to (a) confirm the runtime-calibration path on the invariant TSC and (b) close the §5 loop by showing the naive baseline **does** recover the hierarchy where `clflush` works. Results below will be populated from `python -m autoecho --method wss --runs 3`.

Table 5: Discovered hierarchy vs. ground truth (Intel — placeholder).

Level	Detected capacity	Median latency	p5–p95	Ground truth	Error
L1 Cache	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*
L2 Cache	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*
L3 Cache	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*
DRAM	—	*TBD*	*TBD*	—	—

Validation accuracy: *TBD*; mean absolute capacity error: *TBD*. Estimator comparison, penalty sensitivity, and figures (memory mountain, model selection) to be inserted from the Intel run.

6.4 AMD x86 — *to be measured*

An AMD (Zen) part likewise exposes L1/L2/L3 with 64-byte lines but a different cache organisation (e.g. larger per-CCX L3), providing a second, independent x86 data point and a further test of architecture-agnosticism.

Table 6: Discovered hierarchy vs. ground truth (AMD — placeholder).

Level	Detected capacity	Median latency	p5–p95	Ground truth	Error
L1 Cache	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*
L2 Cache	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*
L3 Cache	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*
DRAM	—	*TBD*	*TBD*	—	—

Validation accuracy: *TBD*; mean absolute capacity error: *TBD*.

6.5 Cross-platform summary and architecture-agnostic behaviour

The level count is never hard-coded: it is chosen from the data, so it adapts to whatever hierarchy the machine exposes. Because K-Means + Silhouette counts distinct latency *levels* directly, the same code path recovers **four** levels on a well-separated x86 L1/L2/L3/DRAM hierarchy, **three** on the Apple M1 (where L2 and the SLC merge), and **two** on a virtualised host whose timer flattens the deeper tiers (Fig. 7). This behaviour was verified on synthetic Intel-, AMD-, M1- and VM-shaped curves: K-Means + Silhouette recovered the correct count on every profile, whereas a fixed PELT penalty over-segmented and a change-point cost-knee under-counted the x86 hierarchy — the empirical reason Silhouette is used as the counter. Physical x86 and Windows runs (§6.3–6.5, §7) remain the step that turns this from *validated in method* into *validated on hardware*.

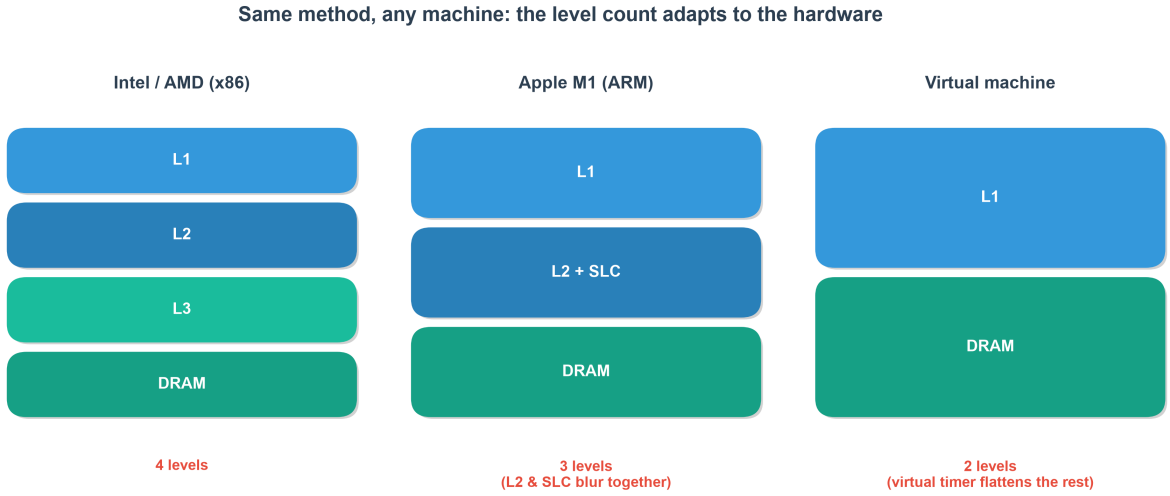

Fig. 7. The same automatic method adapts its level count to the hardware: four levels on x86, three on the Apple M1 (L2 and SLC blur together), two on a virtual machine. The count is chosen from the data, never hard-coded.

Metric	Apple M1 (ARM64)	Intel x86-64	AMD x86-64
Cache line size	128 B	*TBD (64 B)*	*TBD (64 B)*

Metric	Apple M1 (ARM64)	Intel x86-64	AMD x86-64
Levels resolved (automatic)	3 (L1 / L2+SLC / DRAM)	*TBD*	*TBD*
Validation accuracy	100% (2/2)	*TBD*	*TBD*
Mean abs. capacity error	19.9%	*TBD*	*TBD*
Naive baseline (with `clflush`)	n/a (no ARM flush)	*TBD*	*TBD*

A combined cross-machine figure overlaying all three real latency curves (`compare_curves.py`) — the strongest single piece of evidence for the architecture-agnostic claim — will be added once at least the Intel curve exists.

6.6 Threats to validity

Following the structure of the reference paper's own threats section [1]:

- **External validity (generalisation).** Results are from a *single* machine (one Apple M1 performance core). "Cross-platform" and "architecture-agnostic" are therefore claims about the *design* and portable implementation, not yet experimentally demonstrated; the Intel/AMD runs (§6.3–6.5) are required to substantiate them.
- **Construct validity (what is measured).** The pointer chase measures *load-to-use* latency of a serialised dependent chain, which includes base pipeline cost — so the ~1.53 ns L1 figure is not directly comparable to the paper's `rdtscp`-dominated 10–25 ns. Absolute values are best read as *relative* differences between tiers (as the paper itself argues in §6.2).
- **Confounding — TLB and page walks.** As the working set grows, the number of distinct pages touched grows with it; deep-plateau latency therefore includes DTLB-miss and page-walk cost, which page alignment does *not* remove. The mid-cache region (~10–14 MB) cannot yet be cleanly separated into cache vs TLB effects; a huge-page control and performance counters are needed.
- **SLC attribution.** Automatic model selection reports the L2 and SLC as one merged mid-band; the finer split (forced only at an explicit penalty ~ 4) is *consistent with* a distinct L2 and the M1 SLC but is not uniquely attributable to it without per-core-type, cross-core and counter-based experiments.
- **Measurement bias.** Latency is the *minimum* over repeats (a lower envelope that hides variability); and although the sweep order is now seed-randomised to decorrelate size from thermal drift, sustained thermal throttling remains a possible bias on long sweeps.
- **Validation tolerance.** A factor-of-two match is permissive; a 200 KiB detection would still "match" a 128 KiB L1. Ground truth is OS-reported and not fully independent of vendor documentation. Stricter one-to-one matching at multiple tolerances (±10/25/50%) and an external `lmbench` cross-check are planned.
- **Conclusion validity.** With automatic model selection the level count is stable across sweeps and all five estimators agree on three levels (Table 3); the residual run-to-run variation is confined to the less-consistent DBSCAN and GMM counts, which are not used to set the count.

---

7. Discussion & Future Work

Auto-Echo demonstrates accurate, unprivileged, architecture-agnostic hierarchy discovery on Apple Silicon. Remaining directions:

- **x86 Linux and Windows validation.** The framework already builds and runs on Linux and Windows (Section 4.1); the remaining step is execution on physical x86 machines, where `rdtscp` is fine-grained and a documented three-level L1/L2/L3 exists. This is expected to yield a genuine L3 plateau and confirm the runtime-calibration path on the invariant TSC, converting the cross-platform claim from *supported in code* to *empirically demonstrated*. Bare-metal hosts are preferred over virtual machines, whose virtualised timers can flatten the curve.

- **Cross-machine comparison figure.** A helper (`compare_curves.py`) overlays the per-machine latency curves (`wss_curve.csv`) on a single log–log axis with each machine's detected cache boundaries annotated. Comparing the M1's 128 KiB / 12 MiB knees against an x86 machine's distinct L1/L2/L3 knees on one plot provides direct visual evidence for the architecture-agnostic claim.
- **External cross-check against lmbench.** A converter (`crosscheck_lmbench.py`) turns `lat_mem_rd` output [9] into the same curve format, so Auto-Echo's probe can be overlaid against the established tool on identical hardware — validating the measurement against trusted prior art rather than only self-consistency.
- **Second dimension (stride sweep).** Sweeping stride as well as size would recover **cache line size and associativity**, extending Auto-Echo from a 1-D slice to the full memory mountain [11].
- **Automatic model selection (delivered).** Earlier drafts left the PELT penalty as a fixed hyperparameter. It is now removed: the level count is set automatically by Silhouette model selection and the boundaries by penalty-free change-point localisation, so no manual tuning knob remains.
- **Statistical confidence.** Repeated sweeps would let boundaries be reported with confidence intervals rather than point estimates.

---

8. Conclusion

Beginning from a naive echolocation probe whose empirical failure on modern hardware was analysed in detail, this project derived and implemented a principled alternative: a portable, flush-free working-set-size pointer-chase probe with batch-amortised timing, feeding an automatic, penalty-free level-discovery stage in which clustering counts the levels and change-point localises them, validated against live OS ground truth. On an Apple M1 all five estimators agree on three levels and the framework recovers the L1 and L2 capacities to within 0.3 octaves (both documented caches matched within a factor of two); the 12 MiB L2 and the OS-unreported ~8 MB System-Level Cache resolve as a single mid-band, with their finer split a candidate sub-structure that further experiments must confirm (§6.6). Validation on Intel and AMD x86 machines — which expose a documented three-level L1/L2/L3 hierarchy and a working `clflush`, and whose result tables (§6.3–6.5) are laid out ready to populate — is the remaining step to convert the architecture-agnostic claim from *demonstrated on one platform* to *demonstrated across architectures*. On the evidence to date, Auto-Echo is best characterised as a **portable-design framework with an Apple M1 case study**: an accurate, critically evaluated analysis pipeline whose cross-architecture generality is designed-in and awaiting empirical confirmation.

---

9. References

[1] V. Klimis, "Shouting at memory: Where did my write go?" in *Proc. 39th European Conf. on Object-Oriented Programming (ECOOP 2025)*, LIPIcs vol. 333, Art. 41, pp. 41:1–41:25, 2025. doi: 10.4230/LIPIcs.ECOOP.2025.41.

[2] Apple Inc., "mach_absolute_time — Apple developer documentation," 2024. [Online]. Available: https://developer.apple.com/documentation/kernel/1462446-mach_absolute_time

[3] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying density-based local outliers," in *Proc. ACM SIGMOD*, 2000, pp. 93–104.

[4] P. J. Rousseeuw, "Silhouettes: A graphical aid to the interpretation and validation of cluster analysis," *J. Comput. Appl. Math.*, vol. 20, pp. 53–65, 1987.

[5] D. E. Knuth, *The Art of Computer Programming, Vol. 2*, 3rd ed. Addison-Wesley, 1997 (Fisher–Yates/Sattolo shuffle).

[6] C. Truong, L. Oudre, and N. Vayatis, "Selective review of offline change point detection methods," *Signal Processing*, vol. 167, 107299, 2020.

[7] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[8] R. H. Saavedra and A. J. Smith, "Measuring cache and TLB performance and their effect on benchmark

run times," *IEEE Trans. Computers*, vol. 44, no. 10, pp. 1223–1235, 1995.
[9] L. McVoy and C. Staelin, "lmbench: Portable tools for performance analysis," in *Proc. USENIX Annual Technical Conf.*, 1996 (the `lat_mem_rd` pointer-chase benchmark).
[10] K. Yotov, K. Pingali, and P. Stodghill, "Automatic measurement of memory hierarchy parameters," in *Proc. ACM SIGMETRICS*, 2005, pp. 181–192.
[11] R. E. Bryant and D. R. O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed. Pearson, 2015 (the "memory mountain").
[12] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters (DBSCAN)," in *Proc. KDD*, 1996, pp. 226–231.
[13] D. Johnson, "Apple M1 microarchitecture research," 2021. [Online]. Available: https://dougallj.github.io/applecpu/firest
(reverse-engineered Firestorm cache/SLC parameters, corroborating the ~8 MB System-Level Cache).